

Table of Contents

Chapter	Title	Page(s)
Chapter 1	1.1 Abstract	1
	1.2 Introduction	2
Chapter 2	Feasibility Study	3-4
Chapter 3	System Analysis	5-6
Chapter 4	System Specifications 4.1 Hardware Requirements 4.2 Software Requirements	7
Chapter 5	Software Used for Project Development	8-13
Chapter 6	System Design	14-17
Chapter 7	Coding	18-39
Chapter 8	Output Screens	40-42
Chapter 9	Software Testing	43-48
Chapter 10	Conclusion	49
Chapter 11	Future Enhancement	50
Chapter 12	Bibliography	51

CHAPTER: 1

1.1 ABSTRACT

This project focuses on developing a scalable and efficient search engine integrated with user authentication and third-party APIs to deliver dynamic and accurate search results. The system incorporates secure OTP (One-Time Password) verification during user login and registration to enhance security. Once authenticated, users can perform searches through the engine, which fetches relevant data using APIs such as Wikipedia,IMDb and News API. To improve user experience and personalization, the application tracks and stores each user's search history, allowing them to revisit previous queries. The platform is built to handle multiple user sessions efficiently, ensuring smooth performance and responsiveness. Designed with scalability in mind, this project can be easily extended to integrate additional APIs or advanced features like filtering and ranking. Overall, the system provides a secure, user-friendly, and feature-rich search experience suitable for a wide range of applications.

1.2 INTRODUCTION

OBJECTIVE :

The primary objective of this project is to develop and implement a scalable and secure **Snort Search Engine** application that integrates user authentication and dynamic search capabilities using APIs. This system aims to digitize the information retrieval process with features such as OTP-based login, fast data access, and personalized search tracking. It eliminates the need for traditional, manual searching methods and ensures a more secure, efficient, and user-friendly experience.

The key objectives include:

- **To provide a web-based search engine interface** where users can log in securely using OTP authentication and perform fast, accurate searches powered by external APIs.
- **To enable real-time search result fetching** from reliable sources like Wikipedia and IMDb, News app ensuring users receive up-to-date and relevant information.
- **To maintain a search history feature** that allows users to track and revisit their previous queries, enhancing personalization and usability.
- **To ensure secure and error-free user authentication**, eliminating unauthorized access and improving system integrity.
- **To build a scalable backend** that can handle multiple users and large volumes of search queries efficiently.

CHAPTER:2

Feasibility Study

A preliminary investigation has been conducted to assess the feasibility of the **Snort Search Engine** project. The primary aim is to evaluate whether the system will be practically useful, scalable, and sustainable for real-world deployment. This includes assessing the **Technical, Operational, and Economic** feasibility aspects of the system, especially as it incorporates features like OTP authentication, API-based search, and search history tracking. While any system may appear feasible with unlimited resources, this study focuses on delivering realistic, cost-effective solutions within current constraints.

- **Technical Feasibility**
- **Operation Feasibility**
- **Economic Feasibility**

2.1 Technical Feasibility

Key technical concerns during this phase include:

- **Availability of Required Technology:** The technologies needed for developing the Snort Search Engine—such as Python Flask, HTML/CSS, SQLite, and APIs like Wikipedia and IMDb—are readily available and widely supported.
- **System Capability:** The proposed system has the technical capacity to store and manage user data securely, track search history, and handle search operations for multiple users simultaneously.
- **Scalability and Performance:** The system is designed to handle real-time API calls and provide fast search responses regardless of the number or location of users. It can also be upgraded easily for future improvements, such as adding more APIs or implementing advanced search filters.
- **Security and Reliability:** OTP-based authentication, along with session handling and local storage of credentials, ensures data accuracy and access control. The application also prioritizes ease of access and user-friendly navigation.

All development tools are either open-source or already available, ensuring compatibility with current hardware and software infrastructure. Bandwidth and computing resources are sufficient to support the application.

Operational Feasibility

A system is only beneficial if it meets organizational needs and is accepted by its users. For the Snort Search Engine, the following were considered:

- **User and Management Support:** The system has been designed considering user requirements and ease of access. The features like search history, intuitive interface, and login security are aimed at improving user satisfaction.

- **Usability and Adoption:** Given its user-centric design, the system is expected to be used effectively without major resistance. Basic training or onboarding is sufficient for users to navigate and utilize the system's features.
- **Resistance to Change:** Since the platform simplifies the search experience and introduces value-added features (like personalization and history tracking), there is minimal risk of user resistance.

Economic Feasibility

The economic viability of the Snort Search Engine is also favorable:

- **Cost vs Benefit:** The project uses open-source tools and minimal hardware resources, keeping development and deployment costs low. The long-term benefits—including increased productivity, efficient search experience, and secure data handling—outweigh the initial investment.
- **Return on Investment:** The use of modular design allows for future expansion with minimal cost, making it a sustainable and cost-effective solution over time.

CHAPTER:3

SYSTEM ANALYSIS:

INTRODUCTION

Analysis refers to the process of breaking down a system into its components to understand its structure, behaviour, and interrelations. Design involves planning and structuring a system to achieve specific goals effectively and efficiently. **System Analysis and Design** is a combination of methodologies and techniques used to create robust and reliable software solutions that meet user and organizational needs.

For the **Snort Search Engine**, system analysis and design play a vital role in building a secure, scalable, and user-friendly application. The project involves tasks such as understanding user needs, planning functionalities, scheduling development phases, evaluating alternative solutions, performing cost-benefit analysis, and maintaining the system after deployment.

EXISTING SYSTEM

Currently, users rely heavily on general-purpose search engines for retrieving information. These platforms do not offer personalized search features or secure access mechanisms like OTP authentication. There is no centralized system that maintains a history of individual searches or provides a simplified, ad-free, and focused search experience based on trusted APIs.

Additionally, existing solutions do not cater to users who require reliable and accurate data from verified sources such as Wikipedia and IMDb for academic or professional use. Users may face distractions, irrelevant results, or privacy issues.

DRAWBACKS OF EXISTING SYSTEM

- Lack of user authentication and security
- No tracking or history of previous searches
- High chances of irrelevant or misleading search results
- No integration with trusted knowledge sources
- Time-consuming and inefficient for specific data retrieval
- No personalized search experience

PROPOSED SYSTEM

The proposed system, **Snort Search Engine**, is a web-based application designed to provide a secure and personalized search experience. It includes **OTP-based user authentication**, API integration for real-time information from reliable sources like Wikipedia and IMDb, and a feature to track and display user search history. The application streamlines the information retrieval process and reduces user effort while ensuring accurate and relevant results.

This system also addresses data privacy concerns by locally managing user credentials and ensuring that all interactions are logged and secure.

ADVANTAGES OF PROPOSED SYSTEM

- Ensures **user data security** through OTP authentication
- Provides **accurate and relevant search results** from trusted APIs
- Reduces **manual effort** and improves the search experience
- Enables **search history tracking** for better usability
- Offers a **clean and distraction-free interface**
- Improves **response time and efficiency** in retrieving data
- Easily **scalable** to include more APIs or advanced search features in the future

CHAPTER:4

SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS:

- Operating system :Windows 10
- Storage : min 2 GB
- RAM :Recommend min 4GB
- Internet Connection Speed :Higher

4.2 SOFTWARE REQUIREMENTS:

- Front end : HTML,CSS
- Language : python
- Framework :Flask
- DATABASE :SQLite3
- API :Wikipedia, IMDB, News API
- Browser :Mozilla Firefox , Google Chrome
- Processor : intel core i3
- Tool :visual studio, cursor

CHAPTER: 5

Software Used for Project Development:

Python

What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.

Python Indentation

- Indentation refers to the spaces at the beginning of a code line.

- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code.

Python Features:

1. Readability and Simplicity

- **Clear Syntax:** Python's syntax is designed to be readable and straightforward, which enhances code clarity and reduces the learning curve for beginners.
- **Indentation:** Uses indentation to define code blocks instead of braces or keywords, making the code visually organized.

2. Dynamic Typing

- **Flexible Data Types:** Python does not require explicit declaration of data types. Variables can change type dynamically, making it easy to work with different types of data.

3. Interpreted Language

- **Line-by-Line Execution:** Python code is executed line-by-line, which simplifies debugging and allows for quick iterations during development.

4. High-Level Language

- **Abstraction:** Python abstracts many low-level details of the computer's operation, allowing developers to focus on solving problems rather than managing memory or system hardware.

5. Object-Oriented Programming (OOP)

- **Classes and Objects:** Python supports object-oriented programming with features such as classes, inheritance, and encapsulation.
- **Polymorphism and Inheritance:** Allows for the creation of reusable code and the ability to extend existing code.

6. Extensive Standard Library

- **Built-in Modules:** Python includes a rich standard library with modules for file I/O, system operations, networking, web services, and more.
- **Third-Party Libraries:** Python's ecosystem includes a vast collection of third-party libraries available through the Python Package Index (PyPI).

7. Cross-Platform Compatibility

- **Multi-OS Support:** Python is compatible with various operating systems including Windows, macOS, and Linux, allowing code to run across different platforms with minimal modifications.

8. Integrated Exception Handling

- **Error Management:** Python provides built-in mechanisms to handle exceptions and errors using try, except, finally, and raise statements, which helps in building robust applications.

9. Flexible Data Structures

- **Built-in Collections:** Python includes versatile data structures like lists, tuples, sets, and dictionaries, which can be used to manage and manipulate data effectively.

10. Functional Programming Features

- **First-Class Functions:** Functions in Python can be assigned to variables, passed as arguments, and returned from other functions.
- **Lambda Expressions:** Allows for the creation of anonymous functions using the lambda keyword.
- **Higher-Order Functions:** Supports functions like map(), filter(), and reduce() that operate on other functions.

11. Dynamic Memory Management

- **Automatic Garbage Collection:** Python manages memory allocation and deallocation automatically, helping to prevent memory leaks and other related issues.

12. Interactivity

- **REPL (Read-Eval-Print Loop):** Python provides an interactive mode that allows developers to test code snippets and explore Python's features in real-time.

Python Applications:

1. Web Development
2. Data Science and Analytics
3. Machine Learning and Artificial Intelligence
4. Automation and Scripting
5. Software Development
6. Networking and Internet of Things (IoT)
7. Database Management
8. Scientific Computing
9. Education
10. Finance and Economics

FLASK

What is Flask?

Flask is a lightweight and flexible web framework written in Python. It is designed to make web development simple and easy to get started with, while still being powerful enough for full-fledged web applications like the **Snort Search Engine**.

Unlike Django, Flask follows a **micro-framework** approach, meaning it does not include built-in tools like an ORM or authentication system by default. Instead, it gives developers the freedom to

choose their own components based on the project's needs. For Snort, Flask was chosen due to its simplicity, modularity, and control over each element of the application.

Flask makes it easy to handle routes, integrate APIs, manage templates, and connect to databases, all while allowing the developer to maintain clear code structure.

How does Flask Work?

Flask loosely follows the **MVC pattern** (Model-View-Controller), similar to Django's MVT. In Flask:

- **Model** – Represents the structure of data, typically stored in a database like SQLite, using either raw SQL or an ORM like SQLAlchemy.
- **View** – Handles HTTP requests and returns responses by calling templates or returning data.
- **Template** – Renders the HTML content for the browser using dynamic values passed from the view functions.

Model in Snort:

In **Snort**, models are used to manage user data such as registration credentials, OTP verification status, and search history. These are stored in a local SQLite database. We can define models manually using SQL or by using an ORM like SQLAlchemy for better abstraction.

View in Snort:

Flask views in Snort are Python functions associated with specific URLs (routes). These functions handle user actions like:

- Registering a user
- Logging in with OTP verification
- Making a search request
- Displaying search results and history

Each view processes the incoming request, interacts with the model (if needed), and renders the appropriate HTML template.

Template in Snort:

Templates in Snort are .html files located in the templates folder. These files use standard HTML along with **Jinja2 templating** syntax (which Flask supports) to dynamically display data such as:

html

CopyEdit

```
<h1>Welcome, {{ username }}</h1>
```

```
<p>Your previous searches:</p>
```

```
<ul>
```

```
    {% for query in search_history %}
```

```
        <li>{{ query }}</li>
```

```
    {% endfor %}
```


Templates in Snort are responsible for rendering the user interface of pages like the login form, search bar, and results page, ensuring a clean and user-friendly experience.

SQLite3:

SQLite3 is a popular, lightweight, and self-contained database engine that is widely used for applications requiring a simple, file-based database system. Here's a comprehensive overview of SQLite3, including its features, benefits, and how to use it:

Overview of SQLite3:

- **Self-Contained:** SQLite is a single library that is included in the application, meaning there's no need for a separate server process or system.
- **Serverless:** It operates as a library rather than a standalone server, making it easy to embed in applications.
- **Zero Configuration:** There is no configuration needed; it works out of the box with minimal setup.
- **Cross-Platform:** SQLite databases are portable and work across various platforms including Windows, macOS, Linux, iOS, and Android.
- **Lightweight:** The database engine is designed to be small and efficient. The entire database engine is typically less than 1 MB in size.

Features of SQLite3:

- **ACID Compliance:** SQLite provides full support for transactions, ensuring atomicity, consistency, isolation, and durability.
- **SQL Support:** SQLite supports a significant subset of SQL-92 standard.
- **File-Based Storage:** The database is stored in a single cross-platform disk file, which simplifies backup and distribution.
- **Indexing:** Supports indexes to improve query performance.
- **Concurrency:** Uses database-level locking for concurrent access.
- **Self-Repairing:** It has a robust file format that can automatically detect and correct file corruption.

APIs Used in Snort Search Engine

Snort uses several third-party APIs to fetch and display dynamic, relevant information to users. Below are the main APIs integrated and instructions on obtaining their access keys.

1. Wikipedia API

- **Purpose:** Provides access to extensive encyclopedia data, including article summaries and details.
- **Integration:** Wikipedia's API is free and open, requiring no API key. You can directly send requests to their RESTful endpoints.
- **Usage:** Snort sends search queries to the Wikipedia API, fetches JSON data, and presents article summaries to users.

2. IMDb API

- **Purpose:** Supplies detailed movie and TV show information, such as titles, ratings, cast, and summaries.

- **Integration:** Most IMDb APIs require registration to get an API key.
- **How to Get API Key:**
 1. Visit a reliable IMDb API provider (such as [OMDb API](#) or RapidAPI's IMDb services).
 2. Create a free account or sign up.
 3. Generate or request your API key from their dashboard.
 4. Use this key in your HTTP request headers or query parameters to authenticate your requests.
- **Usage:** Snort queries the IMDb API with user search terms, including the API key, to retrieve movie/show details.

3. News API (Optional)

- **Purpose:** Fetches the latest news articles related to search queries.
- **Integration:** Requires registration and an API key.
- **How to Get API Key:**
 1. Go to [NewsAPI.org](#).
 2. Sign up for a free account.
 3. Obtain your API key from the user dashboard.
 4. Include the API key in your requests to fetch news data.
 5. **Usage:** When included, Snort sends search queries to the News API with the key, receiving current news headlines and articles.

CHAPTER:6

SYSTEM DESIGN

The Snort Search Engine is structured into distinct functional modules to ensure maintainability, scalability, and efficient performance. The system is built using Python Flask, SQLite3, and standard web technologies (HTML, CSS), and incorporates the following core modules:

User Registration and Authentication Module

This module allows users to register with the Snort Search Engine platform. Once registered, users can securely log in and gain access to the full suite of search functionalities. It includes:

- User Sign-Up: Collects and stores username, password, and email in the SQLite database.
- Login System: Verifies user credentials securely and grants access to the dashboard.
- OTP Verification: Implements a one-time password mechanism for email verification.
- Session Handling: Tracks user sessions for personalized access.

Search Engine Module

This is the core functional module of Snort. It integrates third-party APIs such as Wikipedia and IMDb to deliver relevant search results to the user.

- Keyword Processing: Handles user search input and sanitizes the query.
- API Integration: Fetches and displays dynamic content using trusted, legal APIs.
- Result Formatting: Presents results in a user-friendly interface with summaries, titles, and links.
- Search History: Logs user search queries for future reference (if authenticated).

Admin Module

The administrative module is restricted to authorized personnel and handles backend data and user management:

- User Management: View or remove registered users.
- Search Logs Monitoring: Admin can view all search logs for moderation or insights.
- System Monitoring: Tracks system status, uptime, and logs for performance review.
- Database Controls: Access to backup or update core data stored in the SQLite database.

Software System Attributes

Snort adheres to essential software quality attributes, also referred to as non-functional requirements. These define how the system behaves and ensure reliability, usability, and sustainability of the solution.

1. Reliability

Snort is designed to provide consistently accurate results from third-party APIs and handle user data with precision. The system will maintain operational reliability with minimal downtime and validated error handling. Target uptime is >99.5%.

2. Availability

Snort ensures that the web application is accessible to users at all times. In the event of a service disruption, appropriate error messages are displayed. The backend is equipped to handle multiple user sessions without crashing or freezing.

3. Security

Security is a critical aspect of Snort. Measures include:

- Secure password storage using hashing techniques.
- OTP-based email verification for account activation.
- Restricted access to admin controls.
- Data integrity checks for critical operations.
- Use of HTTPS and validation against malicious inputs.

4. Maintainability

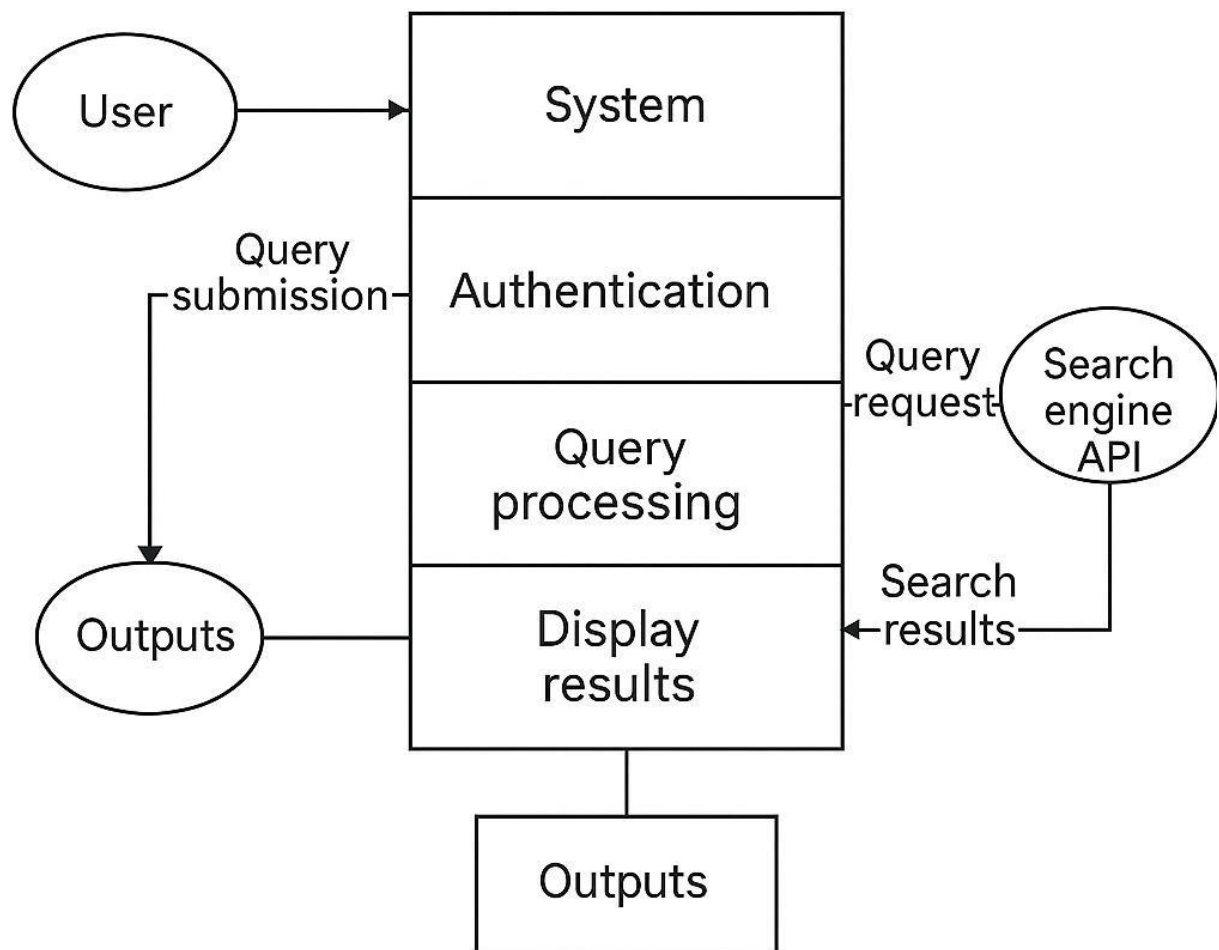
Snort has been developed with modularity and scalability in mind. This makes it easier to update modules independently. Backup mechanisms for user and search data are integrated, and the codebase follows best practices for readability and structure.

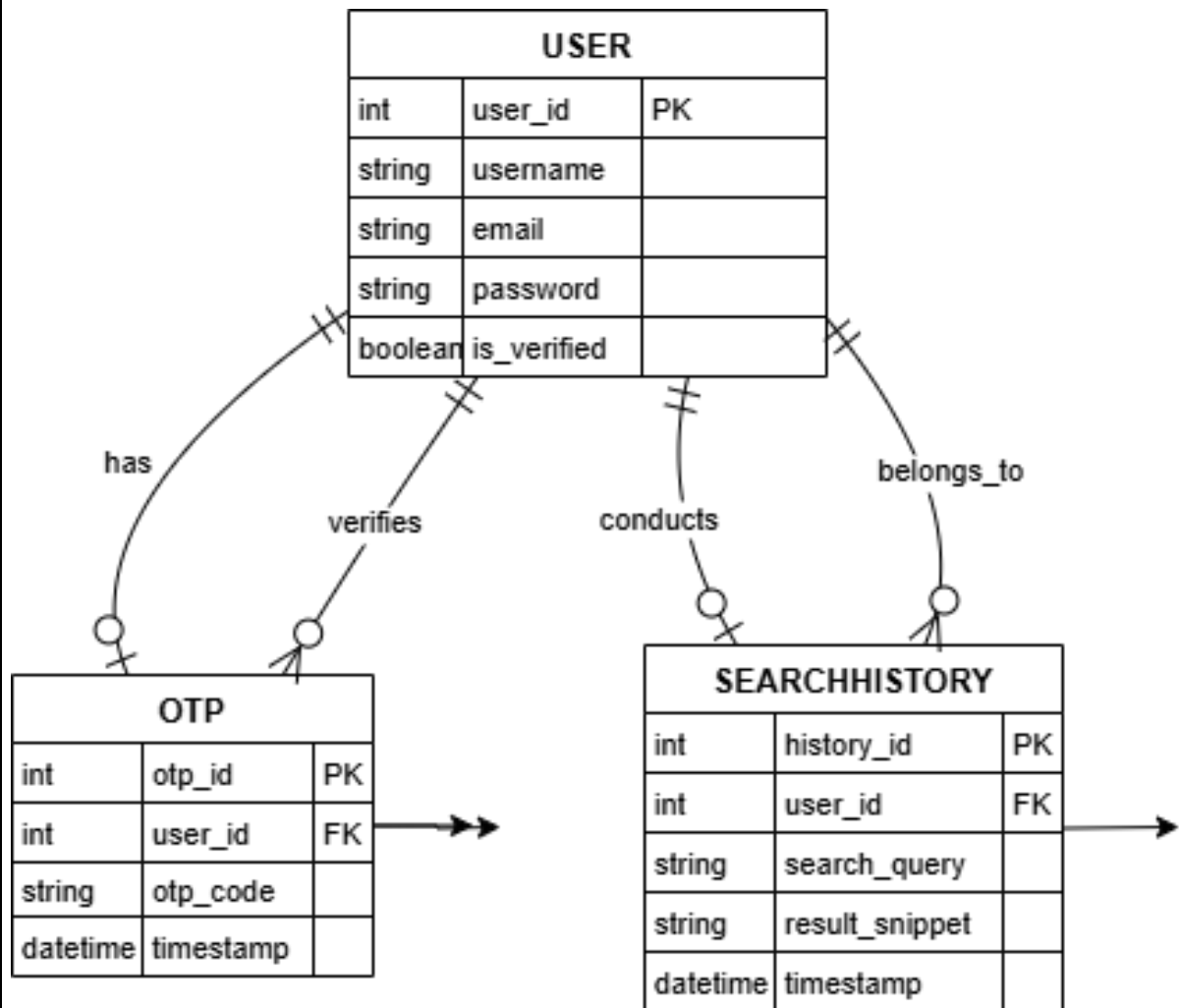
5. Portability

Snort is built using Python Flask and SQLite, both of which are platform-independent. The application can be deployed on any machine running a modern operating system (Windows, Linux, macOS) with minimal configuration.

DATAFLOW DIAGRAM:

Data Flow Diagram for Snort



ER DIAGRAM:

CHAPTER:7

CODING

Project Structure

/snort/

```

├── main.py          # Main application entry point
├── requirements.txt  # Project dependencies
├── read.txt         # This documentation file
├── .gitignore       # Git ignore rules
|
├── static/          # Static assets
|   ├── css/
|   |   └── style.css # Main stylesheet with Apple-inspired design
|
├── login/           # Authentication module
|   ├── app.py       # Login/registration functionality
|   ├── users.db      # User database
|   ├── templates/   # Login-specific templates
|   └── static/       # Login-specific assets
|
└── engine/          # Search engine module
    ├── app.py        # Search functionality
    ├── search_history.db # Search history database
    ├── templates/    # Search-specific templates
    └── static/        # Search-specific assets

```

STATIC.CSS

Style Documentation

Theme Variables

```
```css
```

```
:root {
 --mono-bg: #fff; /* Background color */
 --mono-surface: #f5f5f5; /* Surface/container background */
 --mono-border: #e0e0e0; /* Border color */
 --mono-text: #111; /* Text color */
 --mono-accent: #111; /* Accent color */
 --mono-radius: 12px; /* Border radius */
 --mono-shadow: 0 2px 8px rgba(0,0,0,0.04); /* Box shadow */
 --input-height: 3.2rem; /* Input field height */
}
```

```
```
```

Dark Theme

```
```css
```

```
[data-theme="dark"] {
 --mono-bg: #111;
 --mono-surface: #222;
 --mono-border: #333;
 --mono-text: #fff;
 --mono-accent: #fff;
}
```

```
```
```

Key Components

Container

- Max width: 700px
- Padding: 2.2rem 2rem
- Background: var(--mono-surface)

- Border radius: var(--mono-radius)
 - Box shadow: var(--mono-shadow)
- ### Input Fields
- Height: var(--input-height)
 - Padding: 0 1rem
 - Border radius: var(--mono-radius)
 - Focus state: Border color changes to accent color

Buttons

- Height: var(--input-height)
- Background: var(--mono-accent)
- Color: var(--mono-accent-dark)
- Border radius: var(--mono-radius)
- Hover: Darker background (#222)

Theme Toggle

- Position: Fixed (top-right)
- Size: 2.5rem × 2.5rem
- Border radius: 50%
- Background: var(--mono-bg)

Responsive Design

- Mobile breakpoint: 700px
- Container: 98vw width
- Reduced padding: 1.2rem 0.5rem
- Smaller font sizes: 15px

Common Elements

- Font: 'Inter', system fonts
- Line height: 1.5
- Transitions: 0.2s-0.3s for color/background changes
- Error states: Red border with light background

Base.html

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block title %}SNORT{% endblock %}</title>

</head>

<body>

    <nav class="nav">

        <div class="nav-content">

            <a href="{{ url_for('search.search') }}" class="nav-logo">SNORT</a>

            <div class="nav-links">

                {% if session.get('username') %}

                    <a href="{{ url_for('search.search') }}">Search</a>

                    <a href="{{ url_for('search.search_history') }}">History</a>

                    <a href="{{ url_for('search.profile') }}">Profile</a>

                    <a href="{{ url_for('login.logout') }}">Logout</a>

                {% else %}

                    <a href="{{ url_for('login.login') }}">Login</a>

                    <a href="{{ url_for('login.register') }}">Register</a>

                {% endif %}

            </div>

        </div>

    </nav>

    <main class="main-content">

        <div class="container">

            {% with messages = get_flashed_messages(with_categories=true) %}

                {% if messages %}

                    {% for category, message in messages %}

                        <div class="alert alert-{{ category }}">

                            {{ message }}

                        </div>

                    {% endfor %}

                {% endif %}

            </div>

        </main>

    </body>

</html>

```

```

        {% endfor %}

    {% endif %}

{% endwith %}


{% block content %}{% endblock %}

</div>

</main>

</body>

</html>

```

Index.html

```

{% extends "base.html" %}

{% block title %}Search{% endblock %}

{% block content %}

<div class="card">

    <h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Search</h1>

    <form method="POST" action="{{ url_for('search.search') }}">

        <div class="form-group">

            <input class="form-input" type="text" name="query" placeholder="Enter your search
query..." required>

        </div>

        <button type="submit" class="btn btn-primary">Search</button>

    </form>

</div>

{% if results %}

<div class="search-results">

    {% if results.wikipedia %}

        <div class="search-card">

            <h2 style="margin-bottom: 1rem; font-size: 1.5rem;">Wikipedia</h2>

            <h3 style="margin-bottom: 0.5rem; font-size: 1.2rem;">{{ results.wikipedia.title
}}</h3>

            {% if results.wikipedia.image %}

```

```

        {% endif %}

        <p style="font-size: 0.9rem;">{{ results.wikipedia.summary }}</p>
    </div>

    {% endif %}

    {% if results.omdb %}

        <div class="search-card">

            <h2 style="margin-bottom: 1rem; font-size: 1.5rem;">Movie Information</h2>

            <h3 style="margin-bottom: 0.5rem; font-size: 1.2rem;">{{ results.omdb.title }} ({{
results.omdb.year }})</h3>

            {% if results.omdb.poster %}

            {% endif %}

            <p style="font-size: 0.9rem;">{{ results.omdb.plot }}</p>

        </div>

    {% endif %}

    {% if results.news %}

        <div class="search-card">

            <h2 style="margin-bottom: 1rem; font-size: 1.5rem;">News</h2>

            {% for article in results.news %}

                <div style="margin-bottom: 1.5rem;">

                    <h3 style="margin-bottom: 0.5rem; font-size: 1.1rem;">

                        <a href="{{ article.url }}" style="color: var(--accent-color); text-decoration:
none;">{{ article.title }}</a>

                    </h3>

                    {% if article.image %}

                    {% endif %}

                </div>

            {% endfor %}

        </div>

    {% endif %}

</div>

{% endif %}

{% endblock %}

```

Profile.html

```

{% extends "base.html" %}

{% block title %}Profile{% endblock %}

{% block content %}

<div class="card">

  <h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Profile</h1>

  <div style="margin-bottom: 2rem;">

    <h2 style="margin-bottom: 1rem; font-size: 1.5rem;">Account Information</h2>

    <p style="margin-bottom: 0.5rem;"><strong>Username:</strong> {{ username }}</p>

    <p style="margin-bottom: 0.5rem;"><strong>Email:</strong> {{ email }}</p>

  </div>

  <div>

    <h2 style="margin-bottom: 1rem; font-size: 1.5rem;">Change Password</h2>

    <form method="POST" action="{{ url_for('search.profile') }}">

      <div class="form-group">

        <label class="form-label" for="current_password">Current Password</label>

        <input class="form-input" type="password" id="current_password"
name="current_password" required>

      </div>

      <div class="form-group">

        <label class="form-label" for="new_password">New Password</label>

        <input class="form-input" type="password" id="new_password"
name="new_password" required>

      </div>

      <button type="submit" class="btn btn-primary">Update Password</button>

    </form>

  </div>

  {% if error %}

    <div class="alert alert-error" style="margin-top: 1rem;">

      {{ error }}

    </div>

  {% endif %}

  {% if success %}

    <div class="alert alert-success" style="margin-top: 1rem;">

```

```

        {{ success }}

    </div>

{% endif %}

</div>

{% endblock %}

Search_history.html

{% extends "base.html" %}

{% block title %}Search History{% endblock %}

{% block content %}

<div class="card">

    <h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Search History</h1>

    {% if history %}

        <div class="search-results">

            {% for query, timestamp in history %}

                <div class="search-card">

                    <h3 style="margin-bottom: 0.5rem; font-size: 1.2rem;">{{ query }}</h3>

                    <p style="font-size: 0.9rem; color: var(--secondary-color);">{{ timestamp }}</p>

                    <form method="POST" action="{{ url_for('search.search') }}" style="margin-top:
1rem;">

                        <input type="hidden" name="query" value="{{ query }}">

                        <button type="submit" class="btn btn-secondary">Search Again</button>

                    </form>

                </div>

            {% endfor %}

        </div>

        <form method="POST" action="{{ url_for('search.clear_history') }}" style="margin-top:
2rem;">

            <button type="submit" class="btn btn-primary">Clear History</button>

        </form>

    {% else %}

        <p style="font-size: 1.1rem; color: var(--secondary-color);">No search history found.</p>

    {% endif %}

</div>

```

```
{% endblock %}
```

app.py

```
from flask import Blueprint, render_template, request, current_app, session, redirect, url_for, flash
import requests
import sqlite3
import os
from functools import wraps

search_blueprint = Blueprint('search', __name__, template_folder='templates')
OMDB_API_KEY = os.getenv("OMDB_API_KEY", "8bd05715")
NEWS_API_KEY = os.getenv("NEWS_API_KEY", "cffb46a71cc245d794f72b7c8f2acacc")
def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'username' not in session:
            flash('Please login to access this page', 'error')
            return redirect(url_for('login.login'))
        return f(*args, **kwargs)
    return decorated_function
def init_db(app=None):
    """Initialize the database"""
    conn = sqlite3.connect("search_history.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            query TEXT NOT NULL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
```

```
conn.close()

def save_search_history(query):
    username = session.get("username")
    if not username:
        return

    conn = sqlite3.connect("search_history.db")
    cursor = conn.cursor()
    cursor.execute("INSERT INTO history (username, query) VALUES (?, ?)", (username, query))
    conn.commit()
    conn.close()

def get_search_history_with_timestamps():
    username = session.get("username")
    if not username:
        return []

    conn = sqlite3.connect("search_history.db")
    cursor = conn.cursor()
    cursor.execute("SELECT query, timestamp FROM history WHERE username = ? ORDER BY id DESC LIMIT 10", (username,))
    history = cursor.fetchall()
    conn.close()
    return history

@search_blueprint.route("/clear_history", methods=["POST"])
@login_required
def clear_history():
    username = session.get("username")
    if not username:
        return "Unauthorized", 403

    conn = sqlite3.connect("search_history.db")
    cursor = conn.cursor()
    cursor.execute("DELETE FROM history WHERE username = ?", (username,))
    conn.commit()
    conn.close()

    flash('Search history cleared successfully', 'success')
```

```
        return redirect(url_for('search.search_history'))
    @search_blueprint.route("/history")
    @login_required
    def search_history():
        history = get_search_history_with_timestamps()
        return render_template("search_history.html", history=history)
    def search_wikipedia(query):
        try:
            url = f"https://en.wikipedia.org/api/rest_v1/page/summary/{query}"
            response = requests.get(url, timeout=5)
            if response.status_code == 200:
                data = response.json()
                return {
                    "title": data.get("title"),
                    "summary": data.get("extract"),
                    "image": data.get("thumbnail", {}).get("source")
                }
        except Exception as e:
            current_app.logger.error(f"Wikipedia search error: {str(e)}")
        return None
    def search_omdb(query):
        try:
            url = f"http://www.omdbapi.com/?t={query}&apikey={OMDB_API_KEY}"
            response = requests.get(url, timeout=5)
            if response.status_code == 200:
                data = response.json()
                if data["Response"] == "True":
                    return {
                        "title": data.get("Title"),
                        "year": data.get("Year"),
                        "poster": data.get("Poster"),
                        "plot": data.get("Plot")
                    }
        
```

```
except Exception as e:

    current_app.logger.error(f'OMDb search error: {str(e)}')

    return None

def search_news(query):

    try:

        url = f'https://newsapi.org/v2/everything?q={query}&apiKey={NEWS_API_KEY}'

        response = requests.get(url, timeout=5)

        if response.status_code == 200:

            data = response.json()

            articles = data.get("articles", []):3

            return [{

                "title": article["title"],

                "url": article["url"],

                "image": article.get("urlToImage")

            } for article in articles]

        except Exception as e:

            current_app.logger.error(f'News search error: {str(e)}')

            return []

@search_blueprint.route("/profile", methods=["GET", "POST"])

@login_required

def profile():

    username = session["username"]

    conn = sqlite3.connect("login/users.db")

    cursor = conn.cursor()

    cursor.execute("SELECT email FROM users WHERE username = ?", (username,))

    row = cursor.fetchone()

    email = row[0] if row else "Not found"

    if request.method == "POST":

        current_password = request.form.get("current_password")

        new_password = request.form.get("new_password")

        cursor.execute("SELECT password FROM users WHERE username = ?", (username,))

        stored_password = cursor.fetchone()[0]
```

```
if stored_password != current_password:

    flash('Incorrect current password', 'error')

    conn.close()

    return render_template("profile.html", username=username, email=email)

cursor.execute("UPDATE users SET password = ? WHERE username = ?", (new_password,
username))

conn.commit()

conn.close()

flash('Password updated successfully!', 'success')

return render_template("profile.html", username=username, email=email)

conn.close()

return render_template("profile.html", username=username, email=email)

@search_blueprint.route("/", methods=["GET", "POST"])
@login_required
def search():

    search_results = {}

    if request.method == "POST":

        query = request.form["query"].strip()

        if query:

            save_search_history(query)

            search_results = {

                "wikipedia": search_wikipedia(query),

                "omdb": search_omdb(query),

                "news": search_news(query)

            }

            if not any(search_results.values()):

                flash('No results found for your search query', 'error')

            return render_template("index.html", results=search_results)
```

LOGIN MODULE:**Style.css**

```
:root {  
  --mono-bg: #fff;          /* Background color */  
  --mono-surface: #f5f5f5;   /* Surface/container background */  
  --mono-border: #e0e0e0;    /* Border color */  
  --mono-text: #111;         /* Text color */  
  --mono-accent: #111;       /* Accent color */  
  --mono-radius: 12px;       /* Border radius */  
  --mono-shadow: 0 2px 8px rgba(0,0,0,0.04); /* Box shadow */  
  --input-height: 3.2rem;    /* Input field height */  
}  
[data-theme="dark"] {  
  --mono-bg: #111;  
  --mono-surface: #222;  
  --mono-border: #333;  
  --mono-text: #fff;  
  --mono-accent: #fff;  
}  
- Max width: 700px  
- Padding: 2.2rem 2rem  
- Background: var(--mono-surface)  
- Border radius: var(--mono-radius)  
- Box shadow: var(--mono-shadow)  
- Height: var(--input-height)  
- Padding: 0 1rem  
- Border radius: var(--mono-radius)  
- Focus state: Border color changes to accent color  
- Height: var(--input-height)  
- Background: var(--mono-accent)  
- Color: var(--mono-accent-dark)  
- Border radius: var(--mono-radius)  
- Hover: Darker background (#222)
```

- Position: Fixed (top-right)
- Size: 2.5rem × 2.5rem
- Border radius: 50%
- Background: var(--mono-bg)
- Mobile breakpoint: 700px
- Container: 98vw width
- Reduced padding: 1.2rem 0.5rem
- Smaller font sizes: 15px
- Font: 'Inter', system fonts
- Line height: 1.5
- Transitions: 0.2s-0.3s for color/background changes
- Error states: Red border with light background

Login.html

```
{% extends "base.html" %}

{% block title %}Login{% endblock %}

{% block content %}

<div class="card">

  <h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Login</h1>

  <form method="POST" action="{{ url_for('login.login') }}">

    <div class="form-group">

      <label class="form-label" for="username">Username</label>

      <input class="form-input" type="text" id="username" name="username" required>

    </div>

    <div class="form-group">

      <label class="form-label" for="password">Password</label>

      <input class="form-input" type="password" id="password" name="password" required>

    </div>

    <button type="submit" class="btn btn-primary">Login</button>

  </form>

  <p style="margin-top: 1.5rem; font-size: 0.9rem;">

    Don't have an account? <a href="{{ url_for('login.register') }}" style="color: var(--accent-color); text-decoration: none;">Register here</a>

  </p>

</div>
```

```

</p>
</div>
{% endblock %}

Register.html

{% extends "base.html" %}
{% block title %}Register{% endblock %}
{% block content %}

<div class="card">

  <h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Register</h1>

  <form method="POST" action="{{ url_for('login.register') }}">

    <div class="form-group">

      <label class="form-label" for="username">Username</label>

      <input class="form-input" type="text" id="username" name="username" required>

    </div>

    <div class="form-group">

      <label class="form-label" for="email">Email</label>

      <input class="form-input" type="email" id="email" name="email" required>

    </div>

    <div class="form-group">

      <label class="form-label" for="password">Password</label>

      <input class="form-input" type="password" id="password" name="password" required>

    </div>

    <div class="form-group">

      <label class="form-label" for="confirm_password">Confirm Password</label>

      <input class="form-input" type="password" id="confirm_password"
name="confirm_password" required>

    </div>

    <button type="submit" class="btn btn-primary">Register</button>

  </form>

  <p style="margin-top: 1.5rem; font-size: 0.9rem;">

    Already have an account? <a href="{{ url_for('login.login') }}" style="color: var(--accent-
color); text-decoration: none;">Login here</a>

  </p>

```

```
</div>
```

```
{% endblock %}
```

Verify.html

```
{% extends "base.html" %}
```

```
{% block title %} Verify Email {% endblock %}
```

```
{% block content %}
```

```
<div class="card">
```

```
<h1 style="margin-bottom: 2rem; font-size: 2rem; font-weight: 600;">Verify Your Email</h1>
```

```
<p style="margin-bottom: 2rem; color: var(--secondary-color);">
```

```
    Please enter the OTP sent to your email address to verify your account.
```

```
</p>
```

```
<form method="POST" action="{{ url_for('login.verify', username=username) }}">
```

```
<div class="form-group">
```

```
<label class="form-label" for="otp">OTP Code</label>
```

```
<input class="form-input" type="text" id="otp" name="otp" required
```

```
    placeholder="Enter 6-digit OTP" pattern="[0-9]{6}" maxlength="6">
```

```
</div>
```

```
<button type="submit" class="btn btn-primary">Verify Email</button>
```

```
</form>
```

```
<p style="margin-top: 1.5rem; font-size: 0.9rem;">
```

```
    Didn't receive the OTP? <a href="{{ url_for('login.register') }}"
```

```
        style="color: var(--accent-color); text-decoration: none;">Register again</a>
```

```
</p>
```

```
</div>
```

```
{% endblock %}
```

App.py

```
from flask import Blueprint, render_template, request, redirect, url_for, session, flash
```

```
import sqlite3, random, smtplib, os
```

```
from email.mime.text import MIMEText
```

```
from email.mime.multipart import MIMEMultipart
```

```
from werkzeug.security import generate_password_hash, check_password_hash
```

```
from dotenv import load_dotenv

load_dotenv()

login_blueprint = Blueprint("login", __name__, template_folder="templates",
static_folder="static")

def init_db():

    conn = sqlite3.connect("login/users.db")

    c = conn.cursor()

    c.execute("CREATE TABLE IF NOT EXISTS users (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        username TEXT UNIQUE NOT NULL,

        email TEXT UNIQUE NOT NULL,

        password TEXT NOT NULL,

        verified INTEGER DEFAULT 0,

        otp TEXT

    )")

    conn.commit()

    conn.close()

def send_otp(email, otp):

    sender_email = os.getenv("EMAIL_USER")

    sender_password = os.getenv("EMAIL_PASS")

    if not sender_email or not sender_password:

        print("Email credentials not set in environment variables.")

        return

    subject = "Your Snort OTP Code"

    body = f"Your OTP code is: {otp}"

    msg = MIMEMultipart()

    msg["From"] = sender_email

    msg["To"] = email

    msg["Subject"] = subject

    msg.attach(MIMEText(body, "plain"))

    try:

        server = smtplib.SMTP("smtp.gmail.com", 587)

        server.starttls()
```

```
server.login(sender_email, sender_password)

server.sendmail(sender_email, email, msg.as_string())

server.quit()

except Exception as e:

    print("Error sending email:", e)

@login_blueprint.route("/login", methods=["GET", "POST"])
def login():

    if request.method == "POST":

        username = request.form["username"]

        password = request.form["password"]

        conn = sqlite3.connect("login/users.db")

        c = conn.cursor()

        c.execute("SELECT * FROM users WHERE username = ?", (username,))

        user = c.fetchone()

        conn.close()

        if user and check_password_hash(user[3], password):

            if user[4] == 0: # Check if user is not verified

                flash("Please verify your email first", "error")

                return redirect(url_for("login.verify", username=username))

            session["username"] = username

            flash("Successfully logged in!", "success")

            return redirect(url_for("search.search"))

        else:

            flash("Invalid username or password", "error")

    return render_template("login.html")

@login_blueprint.route("/register", methods=["GET", "POST"])
def register():

    if request.method == "POST":

        username = request.form["username"]

        email = request.form["email"]

        password = request.form["password"]

        confirm_password = request.form["confirm_password"]

        if password != confirm_password:
```

```
flash("Passwords do not match", "error")

return render_template("register.html")

otp = str(random.randint(100000, 999999))

hashed_password = generate_password_hash(password)

conn = sqlite3.connect("login/users.db")

c = conn.cursor()

try:

    c.execute("INSERT INTO users (username, email, password, otp) VALUES (?, ?, ?, ?)",

              (username, email, hashed_password, otp))

    conn.commit()

    send_otp(email, otp)

    flash("Registration successful! Please check your email for OTP verification.", "success")

    return redirect(url_for("login.verify", username=username))

except sqlite3.IntegrityError:

    flash("Username or email already exists", "error")

finally:

    conn.close()

return render_template("register.html")

@login_blueprint.route("/verify/<username>", methods=["GET", "POST"])
def verify(username):

    if request.method == "POST":

        otp_entered = request.form["otp"]

        conn = sqlite3.connect("login/users.db")

        c = conn.cursor()

        c.execute("SELECT otp FROM users WHERE username = ?", (username,))

        user_otp = c.fetchone()

        if user_otp and user_otp[0] == otp_entered:

            c.execute("UPDATE users SET verified = 1, otp = NULL WHERE username = ?",

                      (username,))

            conn.commit()

            conn.close()

            flash("Email verified successfully! Please login.", "success")

            return redirect(url_for("login.login"))
```

```
    else:

        flash("Invalid OTP!", "error")

        return render_template("verify.html", username=username)

    return render_template("verify.html", username=username)

@login_blueprint.route("/logout")
def logout():

    session.pop("username", None)

    flash("Successfully logged out!", "success")

    return redirect(url_for("login.login"))

# Initialize database when the app starts
init_db()

Main.py

from flask import Flask, redirect, url_for, session
from login.app import login_blueprint, init_db as init_login_db
from engine.app import search_blueprint, init_db as init_search_db
from dotenv import load_dotenv
import os

# Load environment variables
load_dotenv()

app = Flask(__name__, template_folder='templates')
app.secret_key = os.getenv("SECRET_KEY",
"32a7f1f51dda2c237090b45699fac699df7e4fcca88cfb1414c8f29a453ad853")

# Register blueprints
app.register_blueprint(login_blueprint)
app.register_blueprint(search_blueprint, url_prefix="/search")

# Initialize databases
init_login_db()
init_search_db()
```

```
@app.route('/')  
def index():  
    if 'username' in session:  
        return redirect(url_for('search.search'))  
    return redirect(url_for('login.login'))  
if __name__ == '__main__':  
    app.run(debug=True)
```

CHAPTER:8

OUTPUT SCREENS

LOGIN PAGE:

Search Engine Login Register

Login

Username

Password

Login

Don't have an account? [Register here](#)

This is login page.

REGISTER PAGE:

Search Engine Login Register

Login

Username

Password

Login

Don't have an account? [Register here](#)

This is registration page to register a user.

VERIFICATION PAGE:

Search Engine

LoginRegister

Registration successfull! Please check your email for OTP verification.

Verify Your Email

Please enter the OTP sent to your email address to verify your account.

OTP Code

Enter 6-digit OTP

Verify Email

Didn't receive the OTP? [Register again](#)

This page verifies the user.

SEARCH PAGE:

Search Engine

SearchHistoryProfileLogout

Successfully logged in!

Search

Enter your search query...

Search

This page used to search the user query.

RESULT PAGE:

Search Engine

SearchHistoryProfileLogout

Search

Enter your search query...

Search

Wikipedia

Google



Google LLC is an American multinational corporation and technology company focusing on online advertising, search engine technology, cloud computing, computer software, quantum computing, e-commerce, consumer electronics, and artificial intelligence (AI). It has been referred to as "the most powerful company in the world" by the BBC and is one of the world's most valuable

News

Google just changed its 'G' logo



Google releases its NotebookLM mobile app



This page shows result for the user query.

SEARCH HISTORY:

Search Engine

SearchHistoryProfileLogout

Search History

INDIA

2025-05-21 13:57:00

Search Again

Clear History

This page shows the user search history

CHAPTER:9

SOFTWARE TESTING

Testing:

Software testing is a method to assess the functionality of the software program. The process checks whether the actual software matches the expected requirements and ensures the software is bug-free. The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements. It mainly aims at measuring the specification, functionality, and performance of a software program or application.

Software testing is an activity performed to identify errors so that errors can be removed to obtain a product with greater quality. To assure and maintain the quality of software and to represent the ultimate review of specification, design, and coding, Software testing is required.

There are different levels of testing:

1.Unit Testing: In this type of testing, errors are detected individually from every component or unit by individually testing the components or units of software to ensure that they are fit for use by the developers. It is the smallest testable part of the software.

2.Integration Testing: In this testing, two or more modules which are unit tested are integrated to test i.e., technique interacting components, and are then verified if these integrated modules work as per the expectation or not, and interface errors are also detected.

3.System Testing: In system testing, complete and integrated Software are tested i.e., all the system elements forming the system are tested as a whole to meet the requirements of the system.

4.Acceptance Testing: This is a kind of testing conducted to ensure that the requirements of the users are fulfilled before its delivery and that the software works correctly in the user's working environment.

Different Types of Software Testing:

1.Manual Testing

2.Automation Testing

There are two different types of software testing currently used in the industry both have their own advantages and disadvantages. If you looking to learn testing from the starting to advance level then you can checkout our dedication software testing course in which we offer very important concept and knowledge you need to master testing.

1. Manual Testing:

Manual testing is a technique to test the software that is carried out using the functions and features of an application. In manual software testing, a tester carries out tests on the software by following a set of predefined test cases. In this testing, testers make test cases for the codes, test the software, and give the final report about that software. Manual testing is time-consuming because it is done by humans, and there is a chance of human errors.

2. Automation Testing:

Automated Testing is a technique where the Tester writes scripts on their own and uses suitable Software or Automation Tool to test the software. It is an Automation Process of a Manual Process. It allows for executing repetitive tasks without the intervention of a Manual Tester.

Types of Manual Testing

- 1.White Box Testing
- 2.Black Box Testing
- 3.Gray Box Testing

1. White Box Testing:

White box testing techniques analyze the internal structures the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing. It is also called glass box testing clear box testing or structural testing. White Box Testing is also known as transparent testing or open box testing.

White box testing is a software testing technique that involves testing the internal structure and workings of a software application. The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.

2. Black Box Testing:

Black-box testing is a type of software testing in which the tester is not concerned with the internal knowledge or implementation details of the software but rather focuses on validating the functionality based on the provided specifications or requirements.

3. Gray Box Testing:

Gray Box Testing is a software testing technique that is a combination of the Black Box Testing technique and the White Box Testing technique.

1.In the Black Box Testing technique, the tester is unaware of the internal structure of the item being tested and in White Box Testing the internal structure is known to the tester.

2.The internal structure is partially known in Gray Box Testing.

3.This includes access to internal data structures and algorithms to design the test cases.

Testing Techniques:**1. Functional Testing:**

- Unit Testing: Tests individual components or functions in isolation. Often automated and written by developers.
- Integration Testing: Checks if different modules or services work together as expected.
- System Testing: Validates the complete and integrated software product against the requirements.
- Acceptance Testing: Ensures the software meets business requirements and is ready for deployment. This can be further divided into Alpha Testing (internal) and Beta Testing (external users).

2. Non-Functional Testing

- Performance Testing: Assesses the speed, responsiveness, and stability of the software under load. Includes:
- Load Testing: Evaluates system behavior under expected user loads.
- Stress Testing: Tests the system under extreme conditions to see how it handles high traffic or data volumes.
- Scalability Testing: Checks if the software can handle growth, such as increasing numbers of users or transactions.
- Usability Testing: Evaluates how user-friendly and intuitive the software is.
- Security Testing: Identifies vulnerabilities and ensures the software is protected against threats. Includes:
- Penetration Testing: Simulates attacks to find security weaknesses.
- Vulnerability Scanning: Identifies known vulnerabilities in the software.

3. Structural Testing:

- White-Box Testing: Focuses on the internal structure of the application. Tests are based on code knowledge, such as:
- Code Coverage: Measures how much of the code is executed during testing.
- Path Testing: Ensures all possible paths through the code are tested.
- Black-Box Testing: Tests the software's functionality without knowledge of the internal code structure. Focuses on input-output behaviour.
- Gray-Box Testing: Combines elements of both white-box and black-box testing. Testers have partial knowledge of the internal structure.

4. Regression Testing:

- Ensures that new code changes do not adversely affect existing functionalities. Often automated and performed after code updates.

5. Exploratory Testing:

- Involves exploring the application without predefined test cases. Testers use their creativity and intuition to find defects that are not covered by formal testing.

6. Automated Testing

- Uses tools and scripts to perform tests automatically. Effective for repetitive testing tasks and large-scale projects. Examples include Selenium for web applications and JUnit for Java applications.

7. Manual Testing:

- Involves human testers performing tests without automation. Useful for exploratory and usability testing where human judgment is crucial.

8. Smoke Testing:

- A preliminary test to check if the major functionalities of the software work. Often referred to as "build verification testing."

9. Sanity Testing:

- Ensures that specific functionalities are working correctly after changes or bug fixes.

10. End-to-End Testing:

- Tests the complete flow of an application from start to finish to ensure the system works as intended.

11. Compatibility Testing:

- Verifies that the software works across different environments, including various browsers, operating systems, and devices.

12. User Acceptance Testing (UAT)

- Performed by end-users to validate that the software meets their needs and requirements before the final release.

Testing Strategies:**1. Test-Driven Development (TDD):**

- Description: Write tests before writing the actual code. The development process is driven by the creation of test cases.
- Focus: Ensures code meets requirements from the outset and helps in writing only the necessary code.
- Benefits: Promotes better design, reduces defects, and improves code coverage.

2. Behaviour-Driven Development (BDD):

- Description: Focuses on the behaviour of the application from the end-user's perspective. Uses natural language to describe test cases.
- Focus: Facilitates communication between developers, testers, and non-technical stakeholders.
- Benefits: Enhances collaboration and ensures the software meets business requirements.

3. Risk-Based Testing:

- Description: Prioritizes testing based on the risk of failure and impact on the business. Focuses on high-risk areas first.
- Focus: Efficiently allocates resources to areas with the highest risk of defects.
- Benefits: Ensures critical functionalities are tested thoroughly and reduces the chance of high-impact failures.

4. Exploratory Testing:

- Description: Testing without predefined test cases. Testers explore the application to find defects.
- Focus: Leveraging tester's creativity and experience to discover issues that structured tests might miss.
- Benefits: Provides a more intuitive understanding of the application and often finds unexpected defects.

5. Automated Testing Strategy:

- Description: Utilizes tools and scripts to perform tests automatically. Suitable for repetitive and regression tests.
- Focus: Efficiency in testing through automation, especially for large-scale projects.
- Benefits: Reduces manual effort, increases test coverage, and speeds up testing cycles.

6. Manual Testing Strategy:

- Description: Involves human testers performing tests without automation. Useful for exploratory and usability testing.
- Focus: Detailed testing where human judgment is crucial.
- Benefits: Allows for nuanced testing and better assessment of user experience.

7. Continuous Testing:

- Description: Integrates testing into the continuous integration/continuous deployment (CI/CD) pipeline. Tests are executed continuously as part of the development process.
- Focus: Ensures that software is tested throughout the development cycle.
- Benefits: Provides rapid feedback on code changes and maintains high quality throughout the development process.

8. Test Coverage Strategy:

- Description: Ensures that all aspects of the codebase are tested. Includes code coverage metrics like line coverage, branch coverage, and function coverage.
- Focus: Comprehensive testing of all code paths and functionalities.
- Benefits: Identifies untested code paths and helps ensure thorough testing.

9. End-to-End Testing Strategy:

- Description: Tests the complete flow of the application from start to finish to ensure all integrated components work together as expected.
- Focus: Validates the overall system functionality and user experience.
- Benefits: Ensures that all components of the system interact correctly and meet user expectations.

10. User Acceptance Testing (UAT) Strategy:

- Description: Conducted by end-users to ensure the software meets their needs and expectations before release.
- Focus: Verifies that the software aligns with business requirements and user needs.
- Benefits: Validates the software from the end-user's perspective and helps ensure it is ready for production.

11. Performance Testing Strategy:

- Description: Includes load testing, stress testing, and scalability testing to assess the performance of the application under various conditions.
- Focus: Ensures the application performs well under expected and extreme conditions.
- Benefits: Identifies performance bottlenecks and ensures the application can handle expected loads.
-

12. Security Testing Strategy:

- Description: Identifies vulnerabilities and assesses the security of the application. Includes penetration testing and vulnerability scanning.
- Focus: Ensures the application is protected against security threats and vulnerabilities.
- Benefits: Protects sensitive data and ensures compliance with security standards.

13. Compatibility Testing Strategy:

- Description: Verifies that the software works across different environments, including various browsers, operating systems, and devices.
- Focus: Ensures the software's compatibility with diverse environments.
- Benefits: Provides a consistent user experience across different platforms and configurations.

14. Regression Testing Strategy:

- Description: Re-tests the application after code changes to ensure that existing functionalities remain unaffected.
- Focus: Ensures that new changes do not introduce defects into existing features.
- Benefits: Maintains the stability of the application as it evolves.

15. Test Data Management Strategy:

- Description: Involves creating and managing test data required for different testing activities.
- Focus: Ensures that accurate and relevant data is available for testing purposes.
- Benefits: Provides realistic test scenarios and improves the accuracy of test result

CHAPTER:10

CONCULSION

The Snort search engine project successfully delivers a scalable and efficient web-based application featuring secure user authentication with OTP verification, dynamic search capabilities, and comprehensive search history tracking. By integrating multiple APIs such as Wikipedia and IMDb, Snort provides users with rich, diverse, and up-to-date information through a seamless and intuitive interface. The use of the Flask framework and SQLite database ensures a robust, maintainable, and high-performance backend capable of handling multiple users concurrently. The project demonstrates effective handling of user credentials, secure API integrations, and reliable data management. Planned future enhancements, such as incorporating additional APIs and improving AI-driven search functionalities, will further elevate Snort's capabilities. Overall, this project highlights the practical application of modern web technologies to build a secure, flexible, and user-friendly search engine that meets the needs of today's users.

CHAPTER:11

FUTURE ENHANCEMENT

Snort's journey as a scalable search engine is just beginning, with immense opportunities to transform it into a next-generation intelligent platform. Imagine a sleek, user-friendly mobile app that puts Snort's powerful search capabilities right in users' pockets—enabling fast, seamless access anytime, anywhere. By integrating AI-driven chatbots, Snort can provide personalized, 24/7 virtual assistance that understands and anticipates user needs, making search interactions smarter and more intuitive.

Expanding the range of APIs to include real-time news feeds, academic databases, social media, and even multimedia content will enrich search results, delivering a truly holistic information experience. Advanced features like voice-activated search and multi-language support will break down barriers, making Snort accessible to a global audience.

Behind the scenes, a dynamic admin dashboard equipped with powerful analytics will provide deep insights into user behavior and system performance, guiding continuous improvements. To safeguard this growing ecosystem, Snort will adopt cutting-edge security protocols like multi-factor authentication and end-to-end encryption, ensuring user data stays protected.

With these innovations, Snort will not just be a search engine—it will be a smart, adaptive, and personalized information hub tailored to meet the evolving demands of modern users worldwide.

CHAPTER:12**BIBLIOGRAPHY**

- "Learning Python, 5th Edition" by Mark Lutz
- Fluent Python: Clear, Concise, and Effective Programming" by Luciano Ramalho
- HTML and CSS: Design and Build Websites by jon duckett
- YOUTUBE CHANNEL TECH WITH TIM
- APNA COLLEGE YOU TUBE CHANNEL